dual_level#

https://pytorch.org/docs/stable/generated/torch.autograd.forward_ad.dual_lev

Description:

Context-manager for forward AD, where all forward AD computation must occur within the dual_level context. The dual_level context appropriately enters and exit the dual level to controls the current forward AD level, which is used by default by the other functions in this API. We currently don't plan to support nested dual_level contexts, however, so only a single forward AD level is supported. To compute higher-order forward grads, one can use torch.func.jvp().

Function Signature

```
class torch.autograd.forward_ad.dual_level[source]#
```

Code Examples

```
>>> x = torch.tensor([1])
>>> x_t = torch.tensor([1])
>>> with dual_level():
...     inp = make_dual(x, x_t)
...     # Do computations with inp
...     out = your_fn(inp)
...     _, grad = unpack_dual(out)
>>> grad is None
False
>>> # After exiting the level, the grad is deleted
>>> _, grad_after = unpack_dual(out)
>>> grad is None
True
```

Notes & Warnings

NOTE Note The dual_level context appropriately enters and exit the dual level to controls the current forward AD level, which is used by default by the other functions in this API. We currently don't plan to support nested dual_level contexts, however, so only a single forward AD level is supported. To compute higher-order forward grads, one can use torch.func.jvp().

Detailed Documentation

Documentation

Context-manager for forward AD, where all forward AD computation must occur within the dual_level context.

The dual_level context appropriately enters and exit the dual level to controls the current forward AD level, which is used by default by the other functions in this API.

We currently don’t plan to support nested dual_level contexts, however, so only a single forward AD level is supported. To compute higher-order forward grads, one can use torch.func.jvp().

Please see the forward-mode AD tutorial for detailed steps on how to use this API.